

E-Commerce Database System Architecture

A Comprehensive, Beginner-Friendly Technical Blueprint & Code Documentation

Target Audience:	Beginner Computer Science & Software Engineering Students	Database:	PostgreSQL / SQLite Relational DBMS
Primary Language:	Python 3.9+ & PL/pgSQL / SQL	Architecture:	3-Tier Architectural Pattern (UI, Controller, DB)
Focus Areas:	ACID Transactions, Stored Procedures, Referential Integrity, & Defensive Programming		

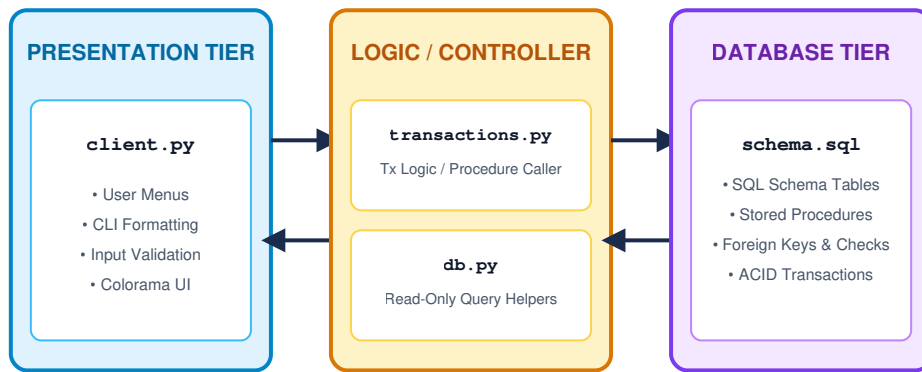
Table of Contents

- 1. System Overview & Architectural Design — Purpose, 3-Tier Model, and Data Flow
- 2. Comprehensive File Guide — Systemic Roles of Every File in the Project
- 3. Database Schema & Data Modeling Analysis (`schema.sql`) — Tables, Constraints, and Indexes
- 4. Data Access Layer Deep Dive (`db.py`) — Connection Management & Read-Only Helpers
- 5. Transaction & Business Logic Engine (`transactions.py`) — Atomic Execution & Stored Procedures
- 6. Presentation & Command-Line User Interface (`client.py`) — Terminal UI & User Workflows
- 7. Theoretical & Practical Computer Science Discussion Questions — ACID, Concurrency, and Snapshots
- 8. Beginner's Implementation & Testing Walkthrough — Step-by-Step Execution Guide

1. System Overview & Architectural Design

The **ShopDB E-Commerce Mini-Project** is an educational software architecture designed to demonstrate how modern, enterprise-grade relational database management systems (RDBMS) process financial transactions safely, reliably, and efficiently. At its core, the application simulates a real-world electronic commerce platform where registered users can log in, browse products, maintain a shopping cart, purchase products using stored payment cards, and request refunds for previous purchases.

For a beginner student in computer science, understanding this project requires looking beyond the raw lines of Python code and visualizing the software as a **3-Tier Architecture**. In software engineering, separating responsibilities into distinct layers ensures that code remains clean, maintainable, secure, and scalable.



The system works as follows:

1. **User Interface Tier (`client.py`)**: Accepts user inputs from the terminal, renders colorful menus using the `colorama` package, maintains an in-memory shopping cart, and displays database query results. It never executes SQL directly.
2. **Application & Logic Tier (`transactions.py` & `db.py`)**: Acts as the intermediary bridge. `db.py` executes simple, safe read-only queries (SELECTs) to fetch product catalogs or order history. `transactions.py` handles complex, multi-step write operations (like checkout or refund) by invoking atomic transactions or stored database procedures.
3. **Database Tier (`schema.sql` / PostgreSQL / SQLite)**: The physical data store. It enforces business rules through column constraints (e.g., `CHECK (stock >= 0)`), foreign key constraints, and stored procedures that guarantee database consistency regardless of client behavior.

Key Computer Science Concept: Separation of Concerns

By decoupling the User Interface (`client.py`) from the Database Logic (`schema.sql`), you could replace the terminal command-line interface with a modern web frontend (like React or Vue) or a mobile application without changing a single line of SQL or transaction logic!

2. Comprehensive File Guide

The project codebase consists of four main files. Each file serves a distinct, non-overlapping architectural role in the operational lifecycle of the system:

File Name	Developer Task	Architectural Role	Summary Description
<code>schema.sql</code>	Student Task 1	Database Tier	Defines the relational schema, tables (<code>users</code> , <code>cards</code> , <code>products</code> , <code>orders</code> , <code>order_items</code>), primary and foreign keys, default values, integrity check constraints, performance indexes, seed data, and server-side stored procedures.
<code>transactions.py</code>	Student Task 2	Controller / Transaction Tier	Exposes high-level transaction operations (<code>checkout</code> and <code>refund_order</code>) to the client. It converts Python data structures (like dicts) into JSON payloads, executes

File Name	Developer Task	Architectural Role	Summary Description
			database calls, manages transaction commit/rollback, and catches exceptions safely.
db.py	Provided (Read-Only)	Data Access Layer (DAL)	Establishes TCP/IP database connection settings via <code>psycopg2</code> , provides connection object factories, and houses helper functions for read-only SQL <code>SELECT</code> queries.
client.py	Provided (Read-Only)	Presentation Layer	A rich Command Line Interface (CLI) application loop that captures user input, displays active cart contents, renders order history, and manages interactive prompts.

3. Database Schema & Data Modeling Analysis (`schema.sql`)

The database schema is the bedrock of any software application. In `schema.sql`, we define the relational structure required to store e-commerce entities and enforce data integrity at the database level.

3.1 Table Entity Relationship Analysis

- **`users` Table:** Stores user profile information (User ID, username, email). Primary key is `user_id`.
- **`cards` Table:** Represents stored payment methods owned by users. Contains `card_id`, `user_id` (Foreign Key referencing `users`), card details, and `balance`. Enforces `CHECK (balance >= 0)` to prevent negative balances.
- **`products` Table:** Holds items available for sale. Contains `product_id`, `name`, `description`, `price`, and `stock` quantity. Enforces `CHECK (stock >= 0)` so stock never drops below zero.
- **`orders` Table:** Stores completed purchase headers. Includes `order_id`, `user_id` (FK), total monetary value (`total`), order status (`status`: e.g., 'paid', 'refunded'), and timestamp (`created_at`).
- **`order\_items` Table:** The junction table implementing a 1-to-Many relationship between orders and products. Stores `order_id` (FK), `product_id` (FK), quantity purchased (`quantity`), and the exact historical unit price snapshot (`unit_price`).

3.2 Critical Schema Constraints

A common mistake made by junior developers is relying solely on Python code to validate data. Modern databases enforce rules via schema constraints:

- **Foreign Keys (`FOREIGN KEY ... REFERENCES`):** Ensures referential integrity. An order cannot be created for a non-existent `user_id`, nor can an order item reference a non-existent `product_id`.
- **Check Constraints (`CHECK`):** Guarantees business rules at the database engine level. For instance, if Python code accidentally attempts to decrement a user's balance below zero, the database engine aborts the transaction immediately with a check constraint violation error.
- **Price Snapshots:** Notice that `order_items` stores `unit_price` independently of `products.price`. If a merchant updates a product's price from £10 to £15 next week, historical receipt records in `order_items` must remain £10. Storing the price snapshot prevents historical distortion.

4. Data Access Layer Deep Dive (`db.py`)

The `db.py` file is responsible for opening connections to PostgreSQL (using the `psycopg2` driver) and running read-only queries. Below is a detailed, function-by-function analysis.

`get_connection()`

CORE INFRASTRUCTURE

What it does: Reads environment configuration settings and establishes a new TCP connection to the PostgreSQL database server.

Why it is useful: Centralizes connection management into a single function. If database credentials or host IP addresses change, developers only need to edit this single location or pass environment variables (e.g., `SHOPDB_PASSWORD`).

How it works (Step-by-Step):

1. Reads configuration parameters from the global dictionary `DB_CONFIG`, which uses `os.getenv()` to fall back on safe defaults (e.g., `host='localhost'`, `port=5432`, `dbname='shopdb'`).
2. Calls `psycopg2.connect(**DB_CONFIG)` to open a socket connection to PostgreSQL.
3. Explicitly sets `conn.autocommit = False`. This is a crucial safety mechanism: it ensures that automatic SQL committing is disabled, giving Python total control over explicit transaction boundaries (`BEGIN`, `COMMIT`, `ROLLBACK`).
4. Returns the live `psycopg2` connection object to the caller.

`_query(sql, params=None)`

READ QUERY HELPER

What it does: An internal helper function that executes read-only SQL `SELECT` statements and returns the results as a list of dictionary-like objects.

Why it is useful: Prevents repetitive boilerplate code across all read functions (opening connection, creating cursor, fetching rows, closing connection). Also eliminates SQL injection vulnerabilities by enforcing parameterized query binding.

How it works (Step-by-Step):

1. Calls `get_connection()` to open a fresh, dedicated database connection.
2. Utilizes a Python `try...finally` block to guarantee that the connection is closed even if an unexpected database error occurs.
3. Instantiates a cursor with `cursor_factory=psycopg2.extras.RealDictCursor`. Unlike standard database cursors that return raw tuples (e.g., `(1, 'Laptop', 999.99)`), `RealDictCursor` returns rows as Python dictionaries (e.g., `{'product_id': 1, 'name': 'Laptop', 'price': 999.99}`).
4. Executes the SQL query passing `params` safely to bind variables (using `%s` placeholders).
5. Fetches all resulting rows using `cur.fetchall()` and returns them to the calling function.
6. Closes the connection in the `finally` block to prevent database connection leaks.

fetch_all_products()

DATABASE READ

What it does: Fetches all items from the `products` table that currently have stock greater than zero, sorted alphabetically by name.

Why it is useful: Powers the product catalog view in the user interface, ensuring customers only see items that are actually available for purchase.

How it works (Step-by-Step):

1. Defines a SQL query: `SELECT product_id, name, description, price, stock FROM products WHERE stock > 0 ORDER BY name`.
2. Passes the query to `_query()`.
3. Returns a list of dictionaries containing product details to the user interface.

fetch_product(product_id)

DATABASE READ

What it does: Retrieves detailed information for a single specific product identified by its unique `product_id`.

Why it is useful: Allows the system to verify the existence or price of an individual item without loading the entire product database into memory.

How it works (Step-by-Step):

1. Executes `_query("SELECT * FROM products WHERE product_id = %s", (product_id,))`.
2. Evaluates the result: if a row is returned, it returns the first record `rows[0]`; if no matching product exists, it returns `None`.

fetch_user(username)

DATABASE READ

What it does: Looks up a user account record in the `users` table based on their unique `username` string.

Why it is useful: Used during the client authentication/login sequence to verify whether a user exists and retrieve their internal `user_id`.

How it works (Step-by-Step):

1. Executes a parameterized SELECT query matching the provided `username` string.
2. Returns the dictionary record if found, or `None` if the username does not exist.

fetch_card(user_id)

DATABASE READ

What it does: Retrieves the primary payment card associated with a given user's numeric `user_id` .

Why it is useful: Needed to verify account balance and card ownership before authorizing transactions.

How it works (Step-by-Step):

1. Queries `cards` table filtering by `user_id = %s` with a `LIMIT 1` clause.
2. Returns the card record dictionary or `None` .

fetch_orders_for_user(user_id)

DATABASE READ

What it does: Fetches all past orders submitted by a specific user, ordered chronologically with the newest orders first.

Why it is useful: Populates the "Order History" screen in the UI and provides order IDs for refund processing.

How it works (Step-by-Step):

1. Executes `SELECT * FROM orders WHERE user_id = %s ORDER BY created_at DESC` .
2. Returns a list of order dictionary objects.

fetch_order_items(order_id)

DATABASE READ

What it does: Fetches the individual line items belonging to a specific order, joining with the `products` table to display human-readable product names and calculating line item totals.

Why it is useful: Displays itemized transaction receipts in the client history menu.

How it works (Step-by-Step):

1. Performs an SQL `INNER JOIN` between `order_items (oi)` and `products (p)` on matching `product_id` fields.
2. Computes an inline column expression: `oi.quantity * oi.unit_price AS line_total` .
3. Filters by `WHERE oi.order_id = %s` and returns the list of itemized records.

5. Transaction & Business Logic Engine (`transactions.py`)

The `transactions.py` file contains the core financial logic of the application. It manages ACID transactions to ensure that checkout and refund operations leave the database in a consistent state.

Class TxResult

DATA TRANSFER OBJECT

What it does: A simple value object (Data Transfer Object) that standardizes responses returned from transaction operations back to the client UI.

Why it is useful: Instead of returning raw booleans or throwing unhandled database exceptions that crash the client, transaction functions return a clean `TxResult` object containing a success boolean, a user-friendly text message, and optional response data (such as a generated `order_id`).

Methods breakdown:

- `__init__(self, success: bool, message: str, data=None)` : Initializes the result state.
- `__repr__(self)` : Provides a formatted string representation for developer debugging (e.g., `TxResult(success=True, message='Payment successful.')`).

What it does: Converts an in-memory shopping cart dictionary into a JSON payload, executes the database checkout stored procedure, retrieves the resulting order confirmation, and handles transaction completion or rollback.

Why it is useful: Executes the entire multi-step purchase process (checking inventory, deducting stock, validating card balance, charging the card, inserting order headers, and inserting line items) inside a single atomic unit of work.

How it works (Step-by-Step):

1. **Cart Validation:** Checks if `cart` is empty. If empty, immediately returns `TxResult(False, "Cart is empty.")` without contacting the database.
2. **JSON Serialization:** Transforms the Python cart dictionary (e.g., `{1: 2, 3: 1}`) into a structured JSON string formatted as array objects: `[{"product_id": 1, "qty": 2}, {"product_id": 3, "qty": 1}]`.
3. **Connection Acquisition:** Calls `get_connection()` to open a database connection.
4. **Stored Procedure Execution:** Opens a cursor and executes `CALL checkout(%s, %s::jsonb)` passing `user_id` and `cart_json`.
5. **Database Processing (PL/pgSQL / Stored Logic):** The database engine opens a transaction boundary, iterates through the cart items, verifies stock level availability, updates product quantities, checks card balances, inserts order and line item rows, and updates card balances. If any check fails, the procedure issues a `RAISE EXCEPTION`.
6. **Order Confirmation Query:** If the stored procedure completes without error, executes a SELECT query to retrieve the newly generated `order_id` and total cost: `SELECT order_id, total FROM orders WHERE user_id = %s ORDER BY created_at DESC LIMIT 1`.
7. **Transaction Commit:** Calls `conn.commit()` to permanently write all changes to disk.
8. **Return Success Result:** Returns `TxResult(True, "Payment of £... successful.", data=order_id)`.
9. **Exception & Rollback Handling:** If a `psycopg2.Error` is caught (e.g., insufficient stock or insufficient card balance), executes `conn.rollback()` to revert all database modifications, extracts the error message from `e.diag.message_primary`, and returns `TxResult(False, msg)`.
10. **Cleanup:** Ensures `conn.close()` is executed inside a `finally` block to return the connection resource to the system.

What it does: Calls the `refund_order` stored procedure in the database to reverse a previously paid order, restore stock levels, and refund the monetary total to the user's payment card.

Why it is useful: Provides a safe, transactional mechanism to cancel orders without introducing database corruption or money duplication errors.

How it works (Step-by-Step):

1. Opens a connection via `get_connection()`.
2. Executes `CALL refund_order(%s, %s)` inside a cursor context block.
3. The database stored procedure verifies: (a) order existence, (b) that the order belongs to `user_id`, and (c) that the order status is currently `'paid'`.
4. The stored procedure iterates through all items in `order_items` associated with `order_id`, adding the purchased quantities back to `products.stock`.
5. The procedure adds the order total back to `cards.balance` and updates the order status in `orders` to `'refunded'`.
6. If successful, Python issues `conn.commit()` and returns a success `TxResult`.
7. If validation fails in the procedure (e.g., trying to refund an order belonging to another user or an already refunded order), `psycopg2.Error` triggers `conn.rollback()`, ensuring zero partial state changes occur.
8. Closes connection resources in the `finally` block.

6. Presentation & Command-Line User Interface (`client.py`)

The `client.py` file provides an interactive, terminal-based user interface. It translates user keyboard inputs into calls to `db.py` and `transactions.py` and formats database responses cleanly for terminal presentation.

6.1 Formatting & Terminal Utility Helpers

- `green(s)`, `red(s)`, `cyan(s)`, `yellow(s)`, `bold(s)`: Color utility wrappers. They wrap output strings in ANSI color codes using the `colorama` library to print green success text, red error messages, yellow prompts, and bold headings. Gracefully falls back to plain text if `colorama` is not installed.
- `clear()`: Prints vertical line breaks to keep terminal screens clean during menu navigation.
- `hr(char="-", width=54)`: Renders horizontal separator lines across the terminal for visual clarity.
- `prompt(msg, valid=None)`: Captures user input in a loop. If a set of `valid` strings is provided, it validates the user input and prompts repeatedly until valid input is received.
- `press_enter()`: Pauses program execution until the user hits the Enter key, preventing screens from disappearing instantly.

6.2 Interactive Application Functions

login() -> dict

[UI VIEW](#)

What it does: Prompts the user to enter a username (e.g., 'alice' or 'bob'), queries the database via `fetch_user()`, and authenticates the session.

How it works: Loops continuously until a valid username exists in the database. Returns the user record dictionary to the caller to maintain session state in `main()`.

show_products(cart: dict) -> list

[UI VIEW](#)

What it does: Fetches all active products via `fetch_all_products()` and prints a clean ASCII table displaying product IDs, names, unit prices, available stock, and the quantity currently present in the user's active cart.

How it works: Iterates through product dictionaries, uses string formatting `"{:<4} {:<24} {:>8}..."` to align text columns, cross-references product IDs with the `cart` dictionary, and highlights cart quantities in yellow.

cart_total(cart: dict, products: list) -> float

[UI CALCULATION](#)

What it does: Calculates the monetary total value of all items in the user's active in-memory cart.

How it works: Constructs a dictionary lookup mapping `product_id` to `price`, iterates through `cart.items()`, multiplies unit prices by quantities, and returns the floating-point sum.

show_cart(cart: dict, products: list)

[UI VIEW](#)

What it does: Displays an itemized view of the user's current shopping cart, including line item sub-totals and the grand total.

How it works: Checks if `cart` is empty. If populated, maps product IDs to names and prices, prints formatted receipt rows, and calls `cart_total()` to print the total cost.

show_order_history(user: dict)

[UI VIEW](#)

What it does: Displays a complete purchase history for the logged-in user, including order status, timestamps, total costs, and nested line items.

How it works: Calls `fetch_orders_for_user(user["user_id"])`. For each order, formats status in green ('paid') or red ('refunded'), calls `fetch_order_items(order_id)` to fetch line items, and prints nested indented sub-lists.

checkout_flow(user: dict, cart: dict, products: list) -> dict

UI WORKFLOW

What it does: Guides the user through confirmation of payment, calls `transactions.checkout()`, displays success or error messages, and empties the cart upon successful purchase.

How it works: Checks if cart is non-empty, requests payment confirmation ('y'/n'), displays a processing status message, invokes `checkout()`, renders colored results based on `result.success`, and returns an empty cart dict `{}` on success or unchanged `cart` on failure.

refund_flow(user: dict)

UI WORKFLOW

What it does: Filters the user's order history for refundable ('paid') orders, presents a list to the user, prompts for an Order ID, and executes `refund_order()`.

How it works: Calls `fetch_orders_for_user()`, filters rows where `status == 'paid'`, prompts for target `order_id`, calls `transactions.refund_order()`, and displays the transaction outcome.

main()

APPLICATION ENTRY POINT

What it does: The primary execution loop of the application. Manages user session state, active cart state, and dispatches menu choices.

How it works: Initiates `login()`, initializes an empty `cart = {}` dictionary, and enters an infinite `while True` loop. Displays the product catalog and menu options (``[a] Add`, `[r] Remove`, `[v] View`, `[c] Checkout`, `[h] History`, `[f] Refund`, `[q] Quit``). Parses keyboard input and dispatches execution to the corresponding workflow functions until ``[q]`` triggers `sys.exit(0)`.

7. Theoretical & Practical Computer Science Discussion Questions

To master transactional database systems, beginner students must understand the core theoretical concepts behind data consistency and concurrency control.

Question 1: What would happen if you committed after each SQL statement instead of once at the end?

Answer: Committing after every individual SQL statement violates the fundamental database property of **Atomicity** (the "A" in ACID transactions). Atomicity guarantees that a transaction is treated as a single, indivisible "all-or-nothing" unit of work.

Consider what happens during `checkout()` if each statement is committed immediately:

1. Statement 1: Product stock is decremented in ``products`` (COMMITTED).
2. Statement 2: User card balance is debited in ``cards`` (COMMITTED).
3. Statement 3: An order header row is inserted into ``orders`` (COMMITTED).

4. Statement 4: Inserting into `order\_items` fails due to a network disconnect or database primary key collision (FAILED).

Because Statements 1, 2, and 3 were already committed individually, the database cannot undo them automatically. The user's money has been deducted, inventory has been reduced, but no itemized order receipt exists! By holding all operations inside a single uncommitted transaction block and issuing a single `COMMIT` at the very end, any failure triggers a `ROLLBACK` that cleanly reverts the database back to its exact initial state as if nothing happened.

Question 2: Two users buy the last unit of a product simultaneously — how does SQLite handle this? What would a production database do differently?

Answer:

SQLite Behavior: SQLite uses file-level locking for concurrency control. When User A begins a write operation (e.g., updating stock during checkout), SQLite places an exclusive write-lock on the entire database file (`shop.db`). When User B attempts to checkout at the exact same instant, SQLite rejects User B's request with a `OperationalError: database is locked` exception (or forces User B to wait until a timeout expires). SQLite ensures safety through coarse-grained single-writer serialization.

Production Database Behavior (PostgreSQL / MySQL): Production multi-user databases utilize **Row-Level Locking** and **Multi-Version Concurrency Control (MVCC)**. Instead of locking the whole database, PostgreSQL locks only the specific row in the `products` table representing that product (using statements like `SELECT ... FOR UPDATE`). User A locks the single product row, updates stock from 1 to 0, and commits. User B's transaction pauses briefly for User A's row-lock to release. When User B's transaction resumes, its `CHECK (stock >= 0)` constraint or stock validation clause fails, rolling back User B's transaction cleanly while allowing all other un-related system transactions to continue unhindered.

Question 3: Why is `unit\_price` stored in `order\_items` rather than looked up from `products` at query time?

Answer: This design choice resolves a fundamental conflict between relational database normalization and historical data immutability.

If `order\_items` omitted `unit\_price` and relied on joining with `products.price` at query time, any future price adjustment made by the store owner would instantly alter historical records. For example, if a customer buys a book for £10 today, and the seller increases the book's price to £25 next month, a query computing the customer's past receipt would dynamically calculate £25, corrupting financial history, invoice records, and tax audits.

Storing a static **snapshot** of `unit\_price` in `order\_items` at the moment of transaction execution guarantees that historical financial records remain permanently immutable regardless of future catalog price changes.

Question 4: What isolation level does SQLite use by default, and what anomalies does it prevent?

Answer:

SQLite defaults to the **SERIALIZABLE** transaction isolation level (achieved via its write-locking mechanism). In PostgreSQL, the default isolation level is **READ COMMITTED**, though transactions can be explicitly configured to run under **REPEATABLE READ** or **SERIALIZABLE**.

By operating under high isolation levels, the system prevents major concurrency anomalies:

- **Dirty Reads:** A transaction reads uncommitted data written by another concurrent transaction that later rolls back. High isolation prevents reading uncommitted changes.
- **Non-Repeatable Reads:** A transaction reads a row twice, but another committed transaction modifies that row in between, yielding different values within the same transaction.
- **Phantom Reads:** A transaction re-executes a search query filtering for rows, but another committed transaction inserts new matching rows in the interim.
- **Lost Updates:** Two concurrent transactions read the same stock value (e.g., stock=1) and both write back decremented values (stock=0), overwriting each other's work and accidentally selling 2 items when only 1 existed.

8. Beginner's Implementation & Testing Walkthrough

To set up and run this project on a local development machine, follow these step-by-step instructions:

Prerequisites

Ensure Python 3.9 or higher and PostgreSQL (or SQLite) are installed on your environment.

Step 1: Database Initialization

Execute the initialization schema script to create tables, constraints, and seed data:

```
# For PostgreSQL environments:
psql -U postgres -d shopdb -f schema.sql

# For SQLite test environments:
sqlite3 shop.db < schema.sql
```

Step 2: Verification of Seed Data

Verify that sample products and test user accounts ('alice' with £1,500 balance and 'bob' with £200 balance) were populated correctly:

```
# Query products table
sqlite3 shop.db "SELECT product_id, name, price, stock FROM products;"
```

Step 3: Launching the Application

Run the interactive client application in your terminal:

```
# Set environment variable for database password (if needed)
export SHOPDB_PASSWORD=yoursecretpassword

# Launch Python client
python client.py
```

Step 4: Executing Self-Tests

To run automated assertion checks against transaction functions in `transactions.py`, run:

```
python transactions.py
```

— End of Technical Architectural Documentation —